

Para dominar a "web-data-viz"

Site: [São Paulo Tech School](#)
Curso: 1ADSA - Algoritmos 2025/2
Livro: Para dominar a "web-data-viz"

Impresso por: MATHEUS STRILICHERK .
Data: segunda-feira, 24 nov. 2025, 14:52

Descrição

Este livro contém recomendações de estudos de conceitos essenciais para entender a [web-data-viz](#).

Índice

1. Guia de Estudos sobre conceitos ESSENCIAIS para entender APIs Node.js com Express (como web-data-viz)

- 1.1. Parâmetros e Retorno de Funções
- 1.2. Tipos de Função
- 1.3. Comunicação Entre Arquivos JS - require() e module.exports
- 1.4. Funções de Callback
- 1.5. Promises (ESSENCIAL)
- 1.6. Comunicação entre as camadas de uma API
- 1.7. Requisições HTTP

1. Guia de Estudos sobre conceitos ESSENCIAIS para entender APIs Node.js com Express (como web-data-viz)

Fluxo completo da comunicação com a API **web-data-viz**:



Frontend (HTML) | Integração (HTML ↔ API) | Backend (API) | Dados (BD)

Para entender as comunicações entre essas camadas da API, é essencial dominar os conceitos a seguir (recomendações de estudo).

Resumo do que é ESSENCIAL entender (próximos capítulos):

Capítulo	Conceito	Importância
1.1 / 1.3	Funções	Entradas, saídas e reuso
1.2	module.exports / require	Separar arquivos (rotas, controllers, models)
1.3	Tipos de função	Leitura, estilo e uso de async/await
1.4	Callbacks	Base do then/catch e da lógica assíncrona
1.5	Promises	Requisições, banco de dados, tudo é assíncrono
1.6	Fluxo da API	Entender como os dados saem do HTML, vão para o Node e voltam
1.7	HTTP	Comunicação entre frontend ↔ backend

1.1. Parâmetros e Retorno de Funções

Para que serve

Funções são blocos de código reutilizáveis e **parâmetros** são os mecanismos usados para **receber valores** dentro de uma função, enquanto **return** serve para **retornar um resultado** (ou finalizar a função cedo) para quem invocou a função.

Essenciais para entender como os dados fluem dentro da aplicação e entre módulos.

Resumo

- **Servem para receber valores, retornar valores e interromper a função)**
- **Parâmetros:** entrada de valores dinâmicos na função
- **Return:** saída que a função te retornará ao final da execução
- Uma função **só retorna algo se tiver return**, senão, o retorno é **undefined**
- Após o **return**, a função é **interrompida**

Recomendações de sites para estudo:

- MDN Functions: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Functions>
- W3Schools Functions: https://www.w3schools.com/js/js_functions.asp

1.2. Tipos de Função

Para que servem os diferentes tipos de função?

Cada tipo existe para facilitar um estilo diferente de codificação.

Função nomeada

Uma função declarada com um nome, que é boa para reutilização (basta invocar seu nome) e debugging.

Função anônima

Função que normalmente é usada como callback ou função interna, muitas vezes sem reutilização (apenas dentro daquele fluxo específico onde ela foi declarada).

Função de Seta (Arrow Function)

Sintaxe curta de declaração de função anônima ou nomeada.

É aplicável no mesmo cenário em que se precisa de uma função anônima (deve declarar diretamente onde vai utilizar) ou nomeada (deve armazenar a arrow function em uma variável para referenciá-la), sendo uma sintaxe mais curta e leve por ser menor.

Possui **parâmetros** e **retorno** por padrão, sendo o mais recomendado para o JS atualmente, pois é leve e padronizado no mercado em novas sintaxes (como as versões atuais de React JS, que recomendam o uso de arrow functions, por exemplo).

Função Assíncrona (async)

Permite usar await para esperar a resolução de promessas de maneira assíncrona.

Pode ser usado em funções nomeadas, funções anônimas ou funções de seta (arrow)

Recomendações de sites para estudo:

- **MDN Function Definitions:** <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions>
- **MDN Arrow Functions:** https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Functions/Arrow_functions

1.3. Comunicação Entre Arquivos JS - require() e module.exports

Require e module.exports (CommonJS)

Permite **importar e exportar** funções, objetos ou valores entre arquivos JS.

É a base da organização em camadas (rotas, controller, model, etc.) em aplicações Node.js tradicionais.

Esse conceito permite dividir o código em arquivos menores e organizados.

- **module.exports = ...** → exporta funções/objetos
- **var exemplo = require('xpto')** → importa os itens exportados de '**xpto.js**' e armazena na variável **exemplo**

Usado **em rotas, controllers, models, config etc.**

Recomendações de sites para estudo:

- Medium: <https://medium.com/@jonathanjuliani/nodejs-require-exports-module-exports-entenda-de-vez-9297dcd5654f>
- MDN require/module.exports (exemplo extra - tem muito mais coisas):
https://developer.mozilla.org/en-US/docs/Learn/Server-side/Express_Nodejs/Introduction#modules
- W3Schools Node Modules (exemplo extra - tem muito mais coisas): https://www.w3schools.com/nodejs/nodejs_modules.asp

1.4. Funções de Callback

Para que serve

Callback: uma função que é passada como argumento para outra função e executada em algum outro momento posterior, dentro da função.

Os métodos **then()** e **catch()** são baseados nisso → eles recebem callbacks e executam.

Esse conceito é essencial para:

- Promises
- then/catch
- Eventos
- Funções assíncronas
- Middleware do Express

Recomendações de sites para estudo:

- MDN Callbacks: https://developer.mozilla.org/en-US/docs/Glossary/Callback_function

1.5. Promises (ESSENCIAL)

Para que serve

Promise representa algo que **ainda não aconteceu**, mas **vai acontecer**.

As Promises devem ser tratadas de maneira assíncrona, indicando que deve ser resolvido ou rejeitado em algum momento (mesmo que não seja imediato).

Toda requisição HTTP retorna uma Promise.

- **resolve()** → deu certo → cai no **.then()**
- **reject()** → deu errado → cai no **.catch()**

Quando o frontend (HTML) faz uma requisição para a API, por debaixo dos panos, a obtenção dos dados não acontece "imediatamente".

A comunicação passa por vários processos, que demandam que o cliente espere os dados, para somente depois, obter a resposta.

Por isso, ao realizar uma requisição HTTP, o método **fetch()** recebe uma **Promise** (promessa), de que em algum momento a resposta será solucionada/resolvida, ou será rejeitada (indicando algum erro).

Então, após realizar a busca com **fetch()**, o método **.then()** serve para que, de maneira assíncrona, aguarde a resolução da Promise, para somente depois, continuar com o tratamento dessa resposta.

Caso algum dado da requisição, função na API ou dados do banco estejam incorretos, pode ocasionar algum erro na busca, então, essa Promise não será **resolvida**, mas sim **rejeitada**, indicando que ocorreu um erro durante a requisição.

O método **.catch()** serve para tratar as promessas rejeitadas, onde ele "captura" o erro e executa a função de callback para tratar esse erro.

Assim, quando a API **resolver** uma Promise corretamente, o método **.then()** é executado para tratar a resposta. Mas, caso a Promise seja **rejeitada** (deu algum erro), o método **.catch()** será acionado, para tratar esses erros.

Por que é essencial para APIs?

Porque **tudo é assíncrono**:

- consultas ao BD
- leitura de arquivos
- requisições externas
- processamento pesado

Esses itens demandam de processamentos que não serão imediatos, logo, o frontend/cliente (HTML) deve AGUARDAR a resposta.

Recomendações de sites para estudo:

- MDN Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- W3Schools JS Promise: https://www.w3schools.com/js/js_promise.asp

1.6. Comunicação entre as camadas de uma API

Para que serve

Entender o fluxo evita erros e facilita o debug em caso de erros.

Fluxo atual do web-data-viz:



Frontend (HTML) | Integração (HTML ↔ API) | Backend (API) | Dados (BD)

Para que o frontend (HTML) e o backend (API) se comuniquem, devemos:

- Realizar uma requisição HTTP, que será recebida pela Rota e direcionada para a Controller respectiva daquela Rota, que representa as regras de negócio da aplicação.
- Caso a controller verifique que os dados da requisição estão corretos, aciona a Model, passando esses dados como parâmetro.
- A Model, aciona o banco de dados para realizar a query (***database.executar(instrucaoSql)***), que é retornada para quem o invocou (neste caso, a Controller).
- A Controller recebe de volta os dados que solicitou para a Model e trata esses dados (verifica se está vazio ou se houveram erros, para retornar o status correto, por exemplo).
- Depois de tratados, os dados da Controller são devolvidos para o HTML (***res.send(xpto)***), que originalmente fez a requisição para a API.
- O HTML recebe as respostas dentro do ".then()", acionado logo após a resolução da Promise recebida pelo fetch();

Com isso, o fluxo completo da aplicação é:

- HTML realiza a requisição para a API;
- Rota recebe a requisição e prepara a resposta;
- Controller é acionada recebendo a requisição e resposta;
- Controller aciona a Model, passando os dados da requisição;
- A Model aciona o banco de dados e retorna os dados para a Controller;
- A Controller trata os dados e retorna para o HTML;

Mas como funcionam as requisições HTTP? (PRÓXIMO CAPÍTULO)

1.7. Requisições HTTP

Para que serve

É o mecanismo de comunicação entre frontend ↔ backend.

Toda requisição HTTP possui:

- **método** (GET, POST, PUT, DELETE)
- **parâmetros** (body, route params, query params)
- **retorno** (JSON, HTML, erro)
- **PROMESSA** (aguarda com then/catch ou async/await)

Recomendações de sites para estudo:

- MDN Fetch API: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- W3Schools HTTP: https://www.w3schools.com/tags/ref_httpmethods.asp
- Fetch Spec: <https://fetch.spec.whatwg.org/#fetch-method>